Rain CLI 도구 구현 분석

개요

Rain BitTorrent 클라이언트는 강력한 CLI(Command Line Interface) 도구를 제공하여 서버 모드와 클라이언트 모드로 동작할 수 있습니다. 이 문서는 `main.go` 의 CLI 구조, RPC 클라이언트 구현, 콘솔 UI 구현, 그리고 각 명령어별 처리 로직을 상세히 분석합니다.

CLI 구조 분석 (`main.go`)

1. 애플리케이션 구조

Rain CLI는 urfave/cli 라이브러리를 사용하여 구현되었습니다.

```
var (
app = cli.NewApp()
clt *rainrpc.Client
log = logger.New("rain")
)

func main() {
app.Version = torrent.Version
app.Usage = "BitTorrent client from https://put.io"
app.EnableBashCompletion = true
// 전역 플래그 설정
```

```go
app.Flags = []cli.Flag{
cli.BoolFlag{
Name: "debug,d",
Usage: "enable debug log",
},
// 프로파일링 플래그들...
}
// 전처리 및 후처리 핸들러
app.Before = handleBeforeCommand
app.After = handleAfterCommand
// 명령어 정의
app.Commands = []cli.Command{
// 다양한 명령어들...
}
err := app.Run(os.Args)
if  err != nil {
log.Fatal(err)
}
}
```

2. 주요 명령어 구조

2.1 서버 모드 명령어

```
{
Name: "server",
Usage: "run rpc server and torrent client",
Flags: []cli.Flag{
cli.StringFlag{
Name: "config,c",
Usage: "read config from `FILE`",
Value: "~/rain/config.yaml",
},
},
Action: handleServer,
}
```

2.2 클라이언트 모드 명령어

```
{
Name: "client",
Usage: "send rpc request to server",
Flags: []cli.Flag{
cli.StringFlag{
Name: "url",
Usage: "URL of RPC server",
Value: "http://127.0.0.1:" + strconv.Itoa(torrent.DefaultConfig.RPCPort),
```

```
},
cli.DurationFlag{
Name: "timeout",
Usage: "request timeout",
Value: 10 * time.Second,
},
},
Before: handleBeforeClient,
Subcommands: []cli.Command{
// 다양한 서브커맨드들...
},
}
```

2.3 단독 다운로드 명령어

```
{
Name: "download",
Usage: "download single torrent",
Flags: []cli.Flag{
cli.StringFlag{
Name: "config,c",
Usage: "read config from `FILE`",
Value: "~/rain/config.yaml",
```

```
},
cli.StringFlag{
Name: "torrent,t",
Usage: "torrent file or URI",
Required: true,
},
cli.BoolFlag{
Name: "seed,s",
Usage: "continue seeding after download is finished",
},
},
Action: handleDownload,
}
```

3. 설정 파일 처리

```
func prepareConfig(c *cli.Context) (torrent.Config, error) {
cfg := torrent.DefaultConfig

configPath := c.String("config")
if configPath != "" {
cp, err := homedir.Expand(configPath)
if err != nil {
```

```
return cfg, err
}
b, err := os.ReadFile(cp)
switch {
case os.IsNotExist(err):
if c.IsSet("config") {
return cfg, err
}
log.Noticef("config file not found at %q, using default config", cp)
case err != nil:
return cfg, err
default:
err = yaml.Unmarshal(b, &cfg)
if err != nil {
return cfg, err
}
log.Infoln("config loaded from:", cp)
}
}
return cfg, nil
}
```

RPC 클라이언트 구현 (`rainrpc` 패키지)

1. 클라이언트 구조

```go
// Client is a JSON-RPC 2.0 client for calling methods of a remote Session.
type Client struct {
client *jsonrpc2.Client
httpClient *http.Client
addr  string
}

func  NewClient(addr  string) *Client {
hc := &http.Client{
Timeout: 10 * time.Second,
}
return &Client{
client: jsonrpc2.NewCustomHTTPClient(addr, hc),
httpClient: hc,
addr: addr,
}
}
```

2. 주요 RPC 메서드

2.1 토렌트 관리 메서드

```go
// 토렌트 목록 조회
func (c *Client) ListTorrents() ([]rpctypes.Torrent, error) {
var  reply rpctypes.ListTorrentsResponse
return  reply.Torrents, c.client.Call("Session.ListTorrents", nil, &reply)
}


// 토렌트 파일 추가
func (c *Client) AddTorrent(f io.Reader, options *AddTorrentOptions) (*rpctypes.
b, err := io.ReadAll(f)
if  err != nil {
return  nil, err
}
args := rpctypes.AddTorrentRequest{
Torrent: base64.StdEncoding.EncodeToString(b)
}
if  options != nil {
args.ID = options.ID
args.Stopped = options.Stopped
args.StopAfterDownload = options.StopAfterDownload
args.StopAfterMetadata = options.StopAfterMetadata
}
var  reply rpctypes.AddTorrentResponse
```

```go
return &reply.Torrent, c.client.Call("Session.AddTorrent", args, &reply)
}

// URI로 토렌트 추가 (마그넷 링크, HTTP URL)
func (c *Client) AddURI(uri  string, options *AddTorrentOptions) (*rpctypes.Torr
args := rpctypes.AddURIRequest{URI: uri}
if  options != nil {
args.ID = options.ID
args.Stopped = options.Stopped
args.StopAfterDownload = options.StopAfterDownload
args.StopAfterMetadata = options.StopAfterMetadata
}
var  reply rpctypes.AddURIResponse
return &reply.Torrent, c.client.Call("Session.AddURI", args, &reply)
}
```

2.2 토렌트 제어 메서드

```go
// 토렌트 시작
func (c *Client) StartTorrent(id  string) error {
args := rpctypes.StartTorrentRequest{ID: id}
var  reply rpctypes.StartTorrentResponse
return  c.client.Call("Session.StartTorrent", args, &reply)
```

```go
}

// 토렌트 중지
func (c *Client) StopTorrent(id string) error {
args := rpctypes.StopTorrentRequest{ID: id}
var reply rpctypes.StopTorrentResponse
return c.client.Call("Session.StopTorrent", args, &reply)
}

// 토렌트 제거
func (c *Client) RemoveTorrent(id string, keepData bool) error {
args := rpctypes.RemoveTorrentRequest{ID: id, KeepData: keepData}
var reply rpctypes.RemoveTorrentResponse
return c.client.Call("Session.RemoveTorrent", args, &reply)
}
```

2.3 정보 조회 메서드

```go
// 토렌트 통계 조회
func (c *Client) GetTorrentStats(id string) (*rpctypes.Stats, error) {
args := rpctypes.GetTorrentStatsRequest{ID: id}
var reply rpctypes.GetTorrentStatsResponse
return &reply.Stats, c.client.Call("Session.GetTorrentStats", args, &reply)
}
```

```go
// 세션 통계 조회
func (c *Client) GetSessionStats() (*rpctypes.SessionStats, error) {
args := rpctypes.GetSessionStatsRequest{}
var reply rpctypes.GetSessionStatsResponse
return &reply.Stats, c.client.Call("Session.GetSessionStats", args, &reply)
}

// 피어 목록 조회
func (c *Client) GetTorrentPeers(id string) ([]rpctypes.Peer, error) {
args := rpctypes.GetTorrentPeersRequest{ID: id}
var reply rpctypes.GetTorrentPeersResponse
return reply.Peers, c.client.Call("Session.GetTorrentPeers", args, &reply)
}
```

3. RPC 통신 프로토콜

Rain은 JSON-RPC 2.0 프로토콜을 사용하여 클라이언트-서버 간 통신을 수행합니다.

3.1 요청 형식 예시

```json
{
"jsonrpc": "2.0",
"method": "Session.AddTorrent",
```

```
"params": {
"torrent": "base64_encoded_torrent_data",
"id": "my-torrent-001",
"stopped": false,
"stopAfterDownload": false,
"stopAfterMetadata": false
},
"id": 1
}
```

3.2 응답 형식 예시

```
{
"jsonrpc": "2.0",
"result": {
"torrent": {
"id": "my-torrent-001",
"name": "Example Torrent",
"infoHash": "abcd1234...",
"port": 20001,
"addedAt": "2023-01-01T00:00:00Z"
}
},
"id": 1
```

```
}
```

콘솔 UI 구현 (`internal/console` 패키지)

1. 콘솔 구조

```
type Console struct {
client *rainrpc.Client
columns []string
needStats  bool

// 상태 관리
selectedID  string
selectedTab  int
selectedPage  int
tabAdjust  int

// 데이터 필드
torrents []Torrent
stats rpctypes.Stats
sessionStats rpctypes.SessionStats
trackers []rpctypes.Tracker
peers []rpctypes.Peer
```

```
webseeds []rpctypes.Webseed


// 동시성 제어
m sync.Mutex
updateTorrentsC chan struct{}
updateDetailsC chan struct{}
stopUpdatingTorrentsC chan struct{}
stopUpdatingSessionStatsC chan struct{}
}
```

2. UI 레이아웃 시스템

2.1 메인 레이아웃 함수

```
func (c *Console) layout(g *gocui.Gui) error {
err := c.drawTitle(g)
if err != nil {
return err
}
// 페이지별 처리
switch c.selectedPage {
case torrents:
err = c.drawTorrents(g)
if err != nil {
```

```
return err
}
err = c.drawDetails(g)
if err != nil {
return err
}
_, err = g.SetCurrentView("torrents")
case sessionStats:
err = c.drawSessionStats(g)
if err != nil {
return err
}
_, err = g.SetCurrentView("session-stats")
case help:
err = c.drawHelp(g)
if err != nil {
return err
}
_, err = g.SetCurrentView("help")
}
return err
}
```

2.2 토렌트 목록 표시

```
func (c *Console) drawTorrents(g *gocui.Gui) error {
maxX, maxY := g.Size()
halfY := maxY / 2
split := halfY + c.tabAdjust
// 헤더 뷰 생성
if v, err := g.SetView("torrents-header", -1, 0, maxX, split); err != nil {
if err != gocui.ErrUnknownView {
return err
}
v.Frame = false
fmt.Fprint(v, getHeader(c.columns))
}
// 토렌트 목록 뷰 생성
if v, err := g.SetView("torrents", -1, 1, maxX, split); err != nil {
if err != gocui.ErrUnknownView {
return err
}
v.Frame = false
v.Highlight = true
v.SelBgColor = gocui.ColorGreen
v.SelFgColor = gocui.ColorBlack
```

```go
fmt.Fprintln(v, "loading torrents...")
} else {
v.Clear()
if c.errTorrents != nil {
fmt.Fprintln(v, "error:", c.errTorrents)
return nil
}

// 토렌트 목록 출력
for i, t := range c.torrents {
fmt.Fprint(v, getRow(c.columns, t, i))
}
}
return nil
}
```

2.3 상세 정보 탭 시스템

```go
func (c *Console) drawDetails(g *gocui.Gui) error {
maxX, maxY := g.Size()
halfY := maxY / 2
split := halfY + c.tabAdjust
if v, err := g.SetView("details", -1, split, maxX, maxY); err != nil {
if err != gocui.ErrUnknownView {
```

```
return err
}
v.Wrap = true
fmt.Fprintln(v, "loading details...")
} else {
v.Clear()
// 탭별 제목 설정
switch c.selectedTab {
case general:
v.Title = "General Info"
case stats:
v.Title = "Stats"
case trackers:
v.Title = "Trackers"
case peers:
v.Title = "Peers"
case webseeds:
v.Title = "WebSeeds"
}
// 탭별 내용 출력
switch c.selectedTab {
case general:
FormatStats(&c.stats, v)
```

```
case stats:
b, err := jsonutil.MarshalCompactPretty(c.stats)
if err != nil {
fmt.Fprintln(v, "error:", err)
} else {
fmt.Fprintln(v, string(b))
}
case trackers:
for i, t := range c.trackers {
fmt.Fprintf(v, "#%d %s\n", i+1, t.URL)
fmt.Fprintf(v, " Status: %s, Seeders: %d, Leechers: %d\n",
t.Status, t.Seeders, t.Leechers)
}
case peers:
format := "%2s %21s %7s %8s %6s %s\n"
fmt.Fprintf(v, format, "#", "Addr", "Flags", "Download", "Upload", "Client")
for i, p := range c.peers {
num := fmt.Sprintf("%d", i+1)
var dl, ul string
if p.DownloadSpeed > 0 {
dl = fmt.Sprintf("%d", p.DownloadSpeed/1024)
}
if p.UploadSpeed > 0 {
```

```
ul = fmt.Sprintf("%d", p.UploadSpeed/1024)
}
fmt.Fprintf(v, format, num, p.Addr, flags(p), dl, ul, p.Client)
}
}
}
return  nil
}
```

3. 키보드 바인딩 시스템

```
func (c *Console) keybindings(g *gocui.Gui) {
// 전역 키
_ = g.SetKeybinding("", gocui.KeyCtrlC, gocui.ModNone, c.forceQuit)


// 종료 키
_ = g.SetKeybinding("torrents", 'q', gocui.ModNone, c.forceQuit)
// 네비게이션 키
_ = g.SetKeybinding("torrents", 'j', gocui.ModNone, c.cursorDown)
_ = g.SetKeybinding("torrents", gocui.KeyArrowDown, gocui.ModNone, c.cursorDown)
_ = g.SetKeybinding("torrents", 'k', gocui.ModNone, c.cursorUp)
_ = g.SetKeybinding("torrents", gocui.KeyArrowUp, gocui.ModNone, c.cursorUp)
// 탭 전환 키
```

```
_ = g.SetKeybinding("torrents", 'g', gocui.ModAlt, c.switchGeneral)
_ = g.SetKeybinding("torrents", 's', gocui.ModAlt, c.switchStats)
_ = g.SetKeybinding("torrents", 't', gocui.ModAlt, c.switchTrackers)
_ = g.SetKeybinding("torrents", 'p', gocui.ModAlt, c.switchPeers)
// 토렌트 제어 키
_ = g.SetKeybinding("torrents", gocui.KeyCtrlS, gocui.ModNone, c.startTorrent)
_ = g.SetKeybinding("torrents", gocui.KeyCtrlS, gocui.ModAlt, c.stopTorrent)
_ = g.SetKeybinding("torrents", gocui.KeyCtrlR, gocui.ModNone, c.removeTorrent)
}
```

4. 실시간 업데이트 시스템

```
func (c *Console) updateTorrentsAndDetailsLoop(g *gocui.Gui, stop  chan  struct{
ticker := time.NewTicker(time.Second)
defer  ticker.Stop()

c.triggerUpdateTorrents()
for {
select {
case <-ticker.C:
c.triggerUpdateTorrents()
c.triggerUpdateDetails(false)
case <-c.updateTorrentsC:
```

```
c.updateTorrents(g)
case <-c.updateDetailsC:
go  c.updateDetails(g)
case <-stop:
return
}
}
}


func (c *Console) updateTorrents(g *gocui.Gui) {
rpcTorrents, err := c.client.ListTorrents()
// 토렌트 정렬
sort.Slice(rpcTorrents, func(i, j  int) bool {
a, b := rpcTorrents[i], rpcTorrents[j]
if  a.AddedAt.Equal(b.AddedAt.Time) {
return  a.ID < b.ID
}
return  a.AddedAt.Before(b.AddedAt.Time)
})
// 통계 정보 병렬 수집
if  c.needStats {
inside := c.rowsInsideView(g)
var  wg sync.WaitGroup
```

```go
for _, i := range inside {
if i < len(torrents) {
t := &torrents[i]
wg.Add(1)
go func(t *Torrent) {
t.Stats, _ = c.client.GetTorrentStats(t.ID)
wg.Done()
}(t)
}
}
wg.Wait()
}
// UI 업데이트
c.m.Lock()
c.torrents = torrents
c.errTorrents = err
c.m.Unlock()
g.Update(c.drawTorrents)
}
```

명령어별 처리 로직

1. 서버 모드 (`handleServer`)

```go
func handleServer(c *cli.Context) error {
cfg, err := prepareConfig(c)
if err != nil {
return err
}
// Session 생성 및 시작
ses, err := torrent.NewSession(cfg)
if err != nil {
return err
}
// 종료 신호 대기
ch := make(chan os.Signal, 1)
signal.Notify(ch, syscall.SIGINT, syscall.SIGTERM)
s := <-ch
log.Noticef("received %s, stopping server", s)

return ses.Close()
}
```

2. 단독 다운로드 모드 (`handleDownload`)

```go
func handleDownload(c *cli.Context) error {
arg := c.String("torrent")
```

```go
seed := c.Bool("seed")
cfg, err := prepareConfig(c)
if err != nil {
return err
}
// 현재 디렉토리에 다운로드
cfg.DataDir = "."
cfg.DataDirIncludesTorrentID = false
// Session 생성
ses, err := torrent.NewSession(cfg)
if err != nil {
return err
}
defer ses.Close()
// 토렌트 추가
var t *torrent.Torrent
opt := &torrent.AddTorrentOptions{
StopAfterDownload: !seed,
}
if isURI(arg) {
t, err = ses.AddURI(arg, opt)
} else {
var f *os.File
```

```
f, err = os.Open(arg)
if err != nil {
return err
}
t, err = ses.AddTorrent(f, opt)
f.Close()
}
if err != nil {
return err
}
// 진행 상황 모니터링
ch := make(chan os.Signal, 1)
signal.Notify(ch, syscall.SIGINT, syscall.SIGTERM)
for {
select {
case s := <-ch:
log.Noticef("received %s, stopping torrent", s)
return t.Stop()
case <-time.After(time.Second):
stats := t.Stats()
progress := 0
if stats.Bytes.Total > 0 {
progress = int((stats.Bytes.Completed * 100) / stats.Bytes.Total)
```

```
}
eta := "?"
if stats.ETA != nil {
eta = stats.ETA.String()
}
log.Infof("Status: %s, Progress: %d%%, Peers: %d, Speed: %dK/s, ETA: %s\n",
stats.Status.String(), progress, stats.Peers.Total,
stats.Speed.Download/1024, eta)
case err = <-t.NotifyStop():
return err
}
}
}
```

3. 클라이언트 명령어들

3.1 토렌트 추가 (handleAdd)

```
func handleAdd(c *cli.Context) error {
arg := c.String("torrent")
addOpt := &rainrpc.AddTorrentOptions{
Stopped: c.Bool("stopped"),
StopAfterDownload: c.Bool("stop-after-download"),
StopAfterMetadata: c.Bool("stop-after-metadata"),
```

```
ID: c.String("id"),
}
var resp *rpctypes.Torrent
var err error
if isURI(arg) {
resp, err = clt.AddURI(arg, addOpt)
} else {
f, err := os.Open(arg)
if err != nil {
return err
}
resp, err = clt.AddTorrent(f, addOpt)
f.Close()
}
if err != nil {
return err
}
// JSON 형태로 결과 출력
b, err := prettyjson.Marshal(resp)
if err != nil {
return err
}
os.Stdout.Write(b)
```

```
os.Stdout.WriteString("\n")
return nil
}
```

3.2 토렌트 목록 조회 (handleList)

```
func handleList(c *cli.Context) error {
resp, err := clt.ListTorrents()
if err != nil {
return err
}
b, err := prettyjson.Marshal(resp)
if err != nil {
return err
}
os.Stdout.Write(b)
os.Stdout.WriteString("\n")
return nil
}
```

3.3 통계 조회 (handleStats)

```
func handleStats(c *cli.Context) error {
s, err := clt.GetTorrentStats(c.String("id"))
```

```go
if err != nil {
return err
}
if c.Bool("json") {
b, err := prettyjson.Marshal(s)
if err != nil {
return err
}
os.Stdout.Write(b)
os.Stdout.WriteString("\n")
return nil
}
// 사람이 읽기 쉬운 형태로 출력
console.FormatStats(s, os.Stdout)
return nil
}
```

3.4 콘솔 UI 실행 (`handleConsole`)

```go
func handleConsole(c *cli.Context) error {
columns := strings.Split(c.String("columns"), " ")
con := console.New(clt, columns)
return con.Run()
```

```
}
```

4. 토렌트 파일 관리 명령어들

4.1 토렌트 파일 정보 표시 (handleTorrentShow)

```
func handleTorrentShow(c *cli.Context) error {
f, err := os.Open(c.String("file"))
if err != nil {
return err
}
defer f.Close()

val := make(map[string]any)
err = bencode.NewDecoder(f).Decode(&val)
if err != nil {
return err
}
// pieces 필드는 너무 크므로 요약 표시
if info, ok := val["info"].(map[string]any); ok {
if pieces, ok := info["pieces"].(string); ok {
info["pieces"] = fmt.Sprintf("<<< %d bytes of data >>>", len(pieces))
}
}
```

```
b, err := prettyjson.Marshal(val)
if err != nil {
return err
}
os.Stdout.Write(b)
os.Stdout.WriteString("\n")
return nil
}
```

4.2 InfoHash 계산 (`handleInfoHash`)

```
func handleInfoHash(c *cli.Context) error {
f, err := os.Open(c.String("file"))
if err != nil {
return err
}
defer f.Close()

var metainfo struct {
Info bencode.RawMessage `bencode:"info"`
}
err = bencode.NewDecoder(f).Decode(&metainfo)
if err != nil {
return err
```

```
}
sum := sha1.Sum(metainfo.Info)
fmt.Println(hex.EncodeToString(sum[:]))
return  nil
}
```

아키텍처 특징

1. 모듈화된 설계

- **CLI 레이어**: 사용자 인터페이스와 명령어 파싱
- **RPC 레이어**: 클라이언트-서버 통신 추상화
- **콘솔 레이어**: 실시간 TUI 인터페이스
- **비즈니스 로직 레이어**: 실제 BitTorrent 구현

2. 유연한 실행 모드

- **서버 모드**: 백그라운드에서 RPC 서버로 동작
- **클라이언트 모드**: 원격 서버에 명령 전송
- **단독 모드**: 단일 토렌트 다운로드
- **콘솔 모드**: 실시간 모니터링 UI

3. 확장 가능한 구조

- 새로운 명령어 추가 용이
- 플러그인 형태의 기능 확장 가능

- 다양한 출력 형식 지원 (JSON, 텍스트)

관련 문서

선행 지식

- 01-system-architecture.md - 전체 시스템 구조와 RPC 아키텍처
- 07-01-library-usage-guide.md - 프로그래밍 인터페이스 기초

관련 주제

- 03-01-torrent-lifecycle-process.md - CLI에서 사용하는 토렌트 처리 과정
- 06-02-concurrency-analysis.md - 콘솔 UI의 고루틴 관리

후속 학습

- 04-01-networking-packages-analysis.md - RPC 통신 구현 세부사항
- 06-03-error-handling-recovery-analysis.md - CLI 에러 처리

> **네비게이션:** 이전: 라이브러리 사용법 가이드 | 목차